

1. Đại số tuyến tính

1.1. Số vô hướng (scalar)

Trong cuộc sống hàng ngày chúng ta sẽ gặp rất nhiều số vô hướng (scalar). Giá trị của tiền nhà tháng này mà bạn phải trả cho chủ nhà là một số vô hướng. Bạn vừa thực hiện một bài kiểm tra toán, bạn được 9 điểm thì điểm số này là một số vô hướng,... Tóm lại số vô hướng là một con số cụ thể. Số vô hướng sẽ khác với biến số vì biến số có thể nhận nhiều giá trị trong khi số vô hướng chỉ nhận một giá trị duy nhất. Ví dụ, khi biểu diễn giá nhà y theo diện tích x theo phương trình $y = 20x + 200$ thì các số vô hướng là 20, 200 và các biến là x, y .

Để khởi tạo một số vô hướng, chúng ta sẽ sử dụng tensor

pytorch numpy

```
import torch

a = torch.tensor(20)
b = torch.tensor(200)
print("diện tích x = 50 --> giá nhà y = a*50+b = ", a*50+b)
```

Số vô hướng có thể được coi như hằng số trong một phương trình. Chúng ta có thể thực hiện các phép toán cộng/trừ/nhân/chia với số vô hướng như với hằng số.

1.2. Véc tơ

Véc tơ là một khái niệm cơ bản nhất của toán học. Chúng ta có thể coi véc tơ là một tập hợp nhiều giá trị của số vô hướng. Véc tơ thường biểu diễn một đại lượng cụ thể trên thực tiễn. Ví dụ như diện tích của các căn nhà là một véc tơ, số lượng phòng ngủ cũng là một véc tơ. Véc tơ có độ dài đặc trưng chính bằng số lượng các phần tử trong nó. Để khởi tạo một véc tơ, trong pytorch chúng ta bao các giá trị của nó trong dấu ngoặc vuông.

pytorch numpy

```
x = torch.tensor([1, 1.2, 1.5, 1.8, 2])
x
```

1.2.1. Các thuộc tính của véc tơ

Một véc tơ sẽ có độ dài và định dạng dữ liệu xác định. Ngoài ra nếu coi một biến số là một véc tơ thì trong thống kê mô tả chúng ta sẽ quan tâm tới tổng, trung bình, phương sai, giá trị lớn nhất, nhỏ nhất.

pytorch numpy

☰ Contents

1. Đại số tuyến tính

1.1. Số vô hướng (scalar)

1.2. Véc tơ

1.3. Ma trận

1.3.1. Các ma trận đặc biệt

1.3.2. Các thuộc tính của ma trận

1.3.3. Các phép tính trên ma trận

1.3.4. Ma trận nghịch đảo

1.3.5. Truy cập thành phần

1.4. Tensor

1.5. Tích giữa một ma trận với véc tơ

1.6. Tích vô hướng (inner product)

1.7. Khái niệm chuẩn

1.8. Định thức và các tính chất của định thức

1.9. Tổ hợp tuyến tính và không gian sinh

1.10. Cơ sở của một không gian

1.11. Biến đổi hệ cơ sở của véc tơ

1.12. Range và Null space

1.13. Hạng của ma trận

2. Tóm tắt

3. Bài tập

```
# Độ dài
print("length of vector: ", x.size()) # or len(x)

# Định dạng của véc tơ
print("vector type: ", x.dtype)

# Tổng của các phần tử
print("sum of vector: ", x.sum())

# Trung bình các phần tử
print("mean of vector: ", x.mean())

# Giá trị nhỏ nhất
print("min of vector: ", x.min())

# Giá trị lớn nhất
print("max of vector: ", x.max())
```

1.2.2. Các phép tính trên véc tơ

Chúng ta có thể thực hiện các phép tính trên véc tơ như phép cộng, trừ, tích vô hướng, tích có hướng giữa hai véc tơ... Lưu ý là chúng phải có cùng độ dài. Trong khuôn khổ cuốn sách này, các véc tơ sẽ được ký hiệu là một ký tự chữ thường in đậm như $\mathbf{x}$, $\mathbf{y}$, $\mathbf{z}$. Ngoài ra $\mathbf{x} \in \mathbb{R}^n$ là véc tơ số thực có độ dài n .

pytorch **numpy**

```
x = torch.tensor([1, 2, 1.5, 1.8, 1.9])
y = torch.tensor([1.1, 2.2, 1.2, 1.6, 1.7])
print("x + y: ", x + y)
print("x - y: ", x - y)
print("x * y: ", x * y)
```

Véc tơ có thể thực hiện các phép cộng, trừ, nhân, chia với một số vô hướng. Giá trị thu được là một véc tơ cùng kích thước mà mỗi phần tử của nó là kết quả được thực hiện trên từng phần tử của véc tơ với số vô hướng đó.

pytorch **numpy**

```
x = torch.tensor([1, 2, 1.5, 1.8, 1.9])
print("x + 5: ", x + 5)
print("x - 5: ", x - 5)
print("x * 5: ", x * 5)
```

1.3. Ma trận

Véc tơ là đại lượng một chiều nên nó chỉ có thể biểu diễn cho một biến. Trong trường hợp chúng ta cần biểu diễn cho nhiều biến thì sẽ cần tới đại lượng hai chiều là ma trận. Ma trận được ký hiệu bởi một chữ cái in đậm $\mathbf{A} \in \mathbb{R}^{m \times n}$ là một ma trận số thực có m dòng và n cột.

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

Để xác định một phần tử bất kỳ thuộc dòng thứ i , cột thứ j của ma trận $\mathbf{A}$ ta ký hiệu chúng là $\mathbf{A}_{ij}$. Véc tơ dòng thứ i sẽ là $\mathbf{A}_{i:}$ và véc tơ cột thứ j sẽ là $\mathbf{A}_{:j}$. Để đơn giản hoá ta qui ước $\mathbf{A}_j$ là véc tơ cột j và $\mathbf{A}^{(i)}$ là véc tơ dòng i .

1.3.1. Các ma trận đặc biệt

- Ma trận vuông: Ma trận vuông là ma trận có số dòng bằng số cột. Ma trận vuông rất quan trọng vì khi tìm nghiệm cho hệ phương trình, từ ma trận vuông ta có thể chuyển sang ma trận tam giác. Ma trận vuông cũng là ma trận có thể tính được giá trị định thức. Tóm lại ma trận $\mathbf{A} \in \mathbb{R}^{m \times n}$ vuông nếu $m = n$.
- Ma trận đơn vị: Là ma trận có đường chéo chính bằng 1, các phần tử còn lại bằng 0. Ví dụ ma trận đơn vị kích thước 3×3 được ký hiệu là $\mathbf{I}_3$ có giá trị là:

$$\mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Tóm lại, $\mathbf{A}$ là ma trận đơn vị nếu nó là ma trận vuông và $a_{ij} = 1$ nếu $i = j$ và $a_{ij} = 0$ nếu $i \neq j$.

- Ma trận đường chéo: Là ma trận có các phần tử trên đường chéo chính khác 0 và các phần tử còn lại bằng 0. Ví dụ về ma trận đường chéo:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

- Ma trận chuyển vị: $\mathbf{B}$ là ma trận chuyển vị của $\mathbf{A}$ nếu $b_{ij} = a_{ji}$ với mọi i, j . Dễ hiểu hơn, tức là mọi dòng của ma trận $\mathbf{A}$ sẽ là cột của ma trận $\mathbf{B}$. Ví dụ:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \end{bmatrix}, \mathbf{B} = \begin{bmatrix} 1 & 3 \\ 2 & 2 \\ 3 & 1 \end{bmatrix}$$

Ký hiệu chuyển vị của ma trận $\mathbf{A}$ là $\mathbf{A}^T$

1.3.2. Các thuộc tính của ma trận

Một ma trận được đặc trưng bởi dòng và cột.

pytorch **numpy**

```
import torch
A = torch.tensor([[1, 2, 3],
                  [3, 2, 1]])

# shape của matrix A
A.size()
```

1.3.3. Các phép tính trên ma trận

Hai ma trận có cùng kích thước chúng ta có thể thực hiện các phép cộng, trừ, tích hadamard (hoặc element-wise). Ma trận thu được cũng có cùng kích thước và các phần tử của nó được tính dựa trên các phần tử có cùng vị trí trên cả hai ma trận $\mathbf{A}$ và $\mathbf{B}$.

Tích hadamard hoặc element-wise

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \dots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \dots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \dots & a_{mn}b_{mn} \end{bmatrix}$$

Trên pytorch chúng ta có thể tính tích hadamard của hai ma trận đơn giản như sau:

pytorch **numpy**

```
import torch
A = torch.tensor([[1, 2, 3],
                  [3, 2, 1]])

B = torch.tensor([[2, 1, 2],
                  [1, 3, 0]])

A*B
```

Tương tự với các phép cộng và trừ

pytorch **numpy**

```
print("A-B: \n", A-B)
print("A+B: \n", A+B)
```

Tích thông thường: Tích thông thường giữa hai ma trận $\mathbf{A}$ có kích thước $m \times n$ và $\mathbf{B}$ có kích thước $n \times p$ là một ma trận có kích thước $m \times p$. Ma trận output $\mathbf{C}$ có giá trị tại phần tử $c_{ij} = \mathbf{A}^{(i)}\mathbf{B}_j$ (dòng thứ i của ma trận $\mathbf{A}$ nhân với cột thứ j của ma trận $\mathbf{B}$).

$$\mathbf{A}_{m \times n} \mathbf{B}_{n \times p} = \begin{bmatrix} \mathbf{A}^{(1)}\mathbf{B}_1 & \mathbf{A}^{(1)}\mathbf{B}_2 & \dots & \mathbf{A}^{(1)}\mathbf{B}_p \\ \mathbf{A}^{(2)}\mathbf{B}_1 & \mathbf{A}^{(2)}\mathbf{B}_2 & \dots & \mathbf{A}^{(2)}\mathbf{B}_p \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}^{(m)}\mathbf{B}_1 & \mathbf{A}^{(m)}\mathbf{B}_2 & \dots & \mathbf{A}^{(m)}\mathbf{B}_p \end{bmatrix}_{m \times p}$$

Chắc các bạn còn nhớ $\mathbf{A}^{(i)}$ là véc tơ dòng và $\mathbf{A}_j$ là véc tơ cột.

pytorch **numpy**

Print to PDF ►

```
import torch
A = torch.tensor([[1, 2, 3],
                  [3, 2, 1]])

B = torch.tensor([[2, 1],
                  [1, 3],
                  [1, 1]])

A@B
```

1.3.4. Ma trận nghịch đảo

Cho ma trận vuông $\mathbf{A} \in \mathbb{R}^{n \times n}$, nếu tồn tại một ma trận vuông $\mathbf{B} \in \mathbb{R}^{n \times n}$ sao cho $\mathbf{AB} = \mathbf{I}_n$ thì ta nói rằng ma trận $\mathbf{A}$ *khả nghịch* (invertible) hoặc *không suy biến* (nonsingular). Ma trận $\mathbf{B}$ được gọi là *ma trận nghịch đảo* (inverse matrix) của ma trận $\mathbf{A}$. Ta dễ dàng nhận thấy rằng nếu $\mathbf{AB} = \mathbf{I}_n$ thì

$$\begin{aligned} \det(\mathbf{AB}) &= \det(\mathbf{I}_n) \\ \Leftrightarrow \det(\mathbf{A})\det(\mathbf{B}) &= 1 \end{aligned}$$

Như vậy để ma trận $\mathbf{A}$ khả nghịch thì $\det(\mathbf{A}) \neq 0$. Trong trường hợp không tồn tại ma trận $\mathbf{B}$ thoả mãn điều kiện $\mathbf{AB} = \mathbf{I}$ thì ta nói rằng ma trận $\mathbf{A}$ *không khả nghịch* hoặc bị *suy biến* (singular).

Ma trận nghịch đảo của *ma trận khả nghịch* $\mathbf{A}$ được kí hiệu là $\mathbf{A}^{-1}$ là một ma trận vuông thoả mãn:

$$\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n$$

Ta dễ nhận thấy rằng $\mathbf{A}^{-1} = \mathbf{B}$. Ma trận nghịch đảo thường được sử dụng để giải hệ phương trình tuyến tính. Giả sử một hệ phương trình tuyến tính:

$$\mathbf{Ax} = \mathbf{b} \tag{1}$$

Trong đó ma trận hệ số $\mathbf{A} \in \mathbb{R}^{n \times n}$ có các dòng tương ứng với các hệ số của một phương trình. Véc tơ $\mathbf{x} \in \mathbb{R}^n$ là biến cần tìm. Véc tơ $\mathbf{b}$ đại diện cho hệ số tự do.

Khi đó để tìm nghiệm $\mathbf{x}$ chúng ta tính theo ma trận nghịch đảo $\mathbf{A}^{-1}$:

$$\begin{aligned} \mathbf{Ax} &= \mathbf{b} \\ \Leftrightarrow \underbrace{\mathbf{A}^{-1}\mathbf{A}}_{\mathbf{I}}\mathbf{x} &= \mathbf{A}^{-1}\mathbf{b} \\ \mathbf{Ix} &= \mathbf{A}^{-1}\mathbf{b} \\ \mathbf{x} &= \mathbf{A}^{-1}\mathbf{b} \end{aligned}$$

Một số tính chất liên quan tới ma trận nghịch đảo.

- Nếu ma trận $\mathbf{A}$ là một ma trận vuông nhưng không khả nghịch thì phương trình (1) có vô số nghiệm. Trong trường hợp ma trận $\mathbf{A}$ không vuông và là một ma trận thấp chiều (có số dòng lớn hơn số cột) thì phương trình có thể vô nghiệm.
- Nếu ma trận $\mathbf{A}$ khả nghịch thì ma trận nghịch đảo của nó là $\mathbf{B}$ cũng khả nghịch. Đồng thời ta có tính chất:

$$\mathbf{A}^{-1} = \mathbf{B} \text{ and } \mathbf{B}^{-1} = \mathbf{A}$$

Chứng minh:

Ta có:

$$\mathbf{AB} = \mathbf{I} \quad (2)$$

Nhân $\mathbf{A}^{-1}$ vào bên trái mỗi vế của phương trình (2) ta được:

$$\begin{aligned} \mathbf{AB} &= \mathbf{I} \\ \underbrace{\mathbf{A}^{-1}\mathbf{A}}_{\mathbf{I}}\mathbf{B} &= \underbrace{\mathbf{A}^{-1}\mathbf{I}}_{\mathbf{A}^{-1}} \\ \mathbf{B} &= \mathbf{A}^{-1} \end{aligned}$$

- Nếu ma trận $\mathbf{A}, \mathbf{B}$ đều khả nghịch thì tích của chúng cũng khả nghịch:

$$(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$$

chứng minh:

Nhân mỗi vế của phương trình trên với $\mathbf{AB}$ vào bên phải ta có:

Đối với vế bên trái:

$$(\mathbf{AB})^{-1}(\mathbf{AB}) = \mathbf{I}$$

Đối với vế bên phải:

$$(\mathbf{B}^{-1}\mathbf{A}^{-1})(\mathbf{AB}) = \mathbf{B}^{-1}(\underbrace{\mathbf{A}^{-1}\mathbf{A}}_{\mathbf{I}})\mathbf{B} = \underbrace{\mathbf{B}^{-1}\mathbf{I}}_{\mathbf{B}^{-1}}\mathbf{B} = \mathbf{B}^{-1}\mathbf{B} = \mathbf{I}$$

Từ đó suy ra $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$

Trên numpy để tính ma trận nghịch đảo chúng ta sử dụng hàm `np.linalg.pinv` hoặc `np.linalg.inv`. Hàm `np.linalg.pinv` sẽ bao gồm cả trường hợp $\mathbf{A}$ không khả nghịch, giá trị trả về là một ma trận giả nghịch đảo (*pseudo-inverse matrix*).

pytorch **numpy**

```
import torch

A = torch.tensor([[1, 2, 3],
                  [3, 2, 1],
                  [4, 2, 2]], dtype=torch.float32)

A_inv = torch.linalg.pinv(A)
print(A_inv)
```

1.3.5. Truy cập thành phần

Chúng ta có thể truy cập vào các thành phần của ma trận $\mathbf{A}$ dựa theo các chỉ số slice index. Chúng ta có thể tổng hợp kiến thức về truy cập thành phần trong bản sau: